

JAVA WEB

Unit 4: Servlet



Introduction

- **Cookies**
 - Ý nghĩa của cookie
 - Các thao tác trên cookie
- **Theo dõi phiên làm việc**
 - Thao tác dữ liệu trong session
 - Sử dụng session trong shopping cart
 - Sử dụng session trong ứng dụng online store
- **Bộ lọc – Servlet Filter**
- **Xây dựng ứng dụng thao tác với CSDL**

Lợi ích của Cookies

Cookies được sử dụng để **xác định người sử dụng** trong phiên giao dịch thương mại điện tử.

Ghi nhớ tên người dùng và mật khẩu (**usernames and passwords**). Cookie cho phép người dùng tự động đăng nhập vào một trang web, mang lại sự thuận tiện đáng kể cho người dùng máy tính **không** dùng chung.

Tùy chỉnh các trang web. Các trang web có thể sử dụng cookie để ghi nhớ các tùy chọn của người dùng.

Định hướng việc quảng cáo (**Focusing advertising**). Cookie giúp trang web ghi nhớ những chủ đề nào mà một số người dùng quan tâm và hiển thị quảng cáo có liên quan đến những sở thích đó.

Ghi Cookie xuống client

1. Tạo một đối tượng Cookie gồm tên và giá trị đều ở dạng chuỗi ký tự.
`Cookie cObject = new Cookie("userID", "a1234" );`
2. Thiết lập thời gian tồn tại của cookie (giờ).
`cObject.setMaxAge(60*60*24*7); // One week`
3. Đặt cookie vào header của HTTP response.
`response.addCookie(cObject);`
Nếu quên bước này thì cookie sẽ không được ghi xuống browser!

Đọc cookie từ client

1. Gọi phương thức **request.getCookies** trả về **mảng** các đối tượng cookie;
2. Sử dụng vòng lặp, gọi phương thức **getName** để lấy tên của đối tượng cookie, sau đó gọi phương thức **getValue** để lấy giá trị tương ứng.

```
String cookieName = "userID";
```

```
Cookie[] cookies = request.getCookies();  
if (cookies != null) {  
    for(int i=0; i<cookies.length; i++) {  
        Cookie cookie = cookies[i];  
        if (cookieName.equals (cookie.getName())) {  
            doSomethingWith ( cookie.getValue() );  
        }  
    }  
}
```

Sử dụng Cookies để phát hiện các Visitors mới

```
Cookie[] cookies = request.getCookies();
```

```
if (cookies == null) { // Correct.
```

```
    doStuffForNewbie();
```

```
} else {
```

```
    doStuffForReturnVisitor();
```

```
}
```

UVINA

Thiết lập các tham số: Domain, Path

Phương thức public void `setDomain(String domainPattern)` - public String `getDomain()`:

Dùng để đặt hoặc truy xuất miền mà cookie áp dụng. Thông thường, trình duyệt chỉ trả lại cookie có cùng một tên máy chủ đã gửi cookie. Ví dụ, các cookie được ghi lại từ một servlet ở địa chỉ: **bali.vacations.com** thì thường sẽ không trả về cho trình duyệt ở địa **queensland.vacations.com**. Nếu trang web muốn điều này xảy ra, các servlet phải chỉ định `cookie.setDomain(".vacations.com")`.

Phương thức public void `setPath(String path)` - public String `getPath()`:

Dùng để đặt hoặc nhận đường dẫn mà cookie áp dụng. Nếu không chỉ định đường dẫn, trình duyệt sẽ chỉ trả lại cookie cho các URL bên trong hoặc bên dưới thư mục chứa trang đã gửi cookie. Ví dụ: nếu máy chủ gửi cookie từ địa chỉ **http://ecommerce.site.com/toys/specials.html**, thì trình duyệt sẽ gửi lại cookie khi kết nối đến địa chỉ **http://ecommerce.site.com/toys/bikes/beginners.html**, nhưng không đến địa chỉ **http://ecommerce.site.com/cds/classical.html**. Phương thức `setPath` có thể chỉ định đường dẫn tổng quát hơn. Ví dụ: `cookie.setPath("/")` chỉ định rằng tất cả các trang trên máy chủ sẽ nhận được cookie.

Ví dụ Basic Cookie Utilities

Tìm kiếm cookie theo tên:

```
public static Cookie getCookie(HttpServletRequest request, String cookieName) {  
    Cookie[] cookies = request.getCookies();  
    if (cookies != null) {  
        for(int i=0; i<cookies.length; i++) {  
            Cookie cookie = cookies[i];  
            if (cookieName.equals(cookie.getName()))    return(cookie);  
        }  
    }  
    return(null);  
}
```


Ví dụ - Basic Cookie Utilities

Tạo Long-Lived Cookies

```
public class LongLivedCookie extends Cookie {  
    public static final int SECONDS_PER_YEAR = 60*60*24*365;  
    public LongLivedCookie(String name, String value) {  
        super(name, value);  
        setMaxAge(SECONDS_PER_YEAR);  
    }  
}
```



Thay đổi giá trị trong cookie

Thông thường, cookie chỉ chứa mã **định danh** duy nhất của người dùng: tất cả dữ liệu về người dùng được lưu trữ trong cơ sở dữ liệu — mã định danh người dùng được sử dụng như là một khóa chính để truy cập thông tin người dùng trong CSDL.

Tuy nhiên, nếu chúng ta muốn định kỳ thay đổi giá trị của cookie thì:

- Cách 1. Thay thế **giá trị** cookie trước đó bằng cách ghi lại một cookie cùng tên bằng một giá trị cookie khác.
- Cách 2. Hướng dẫn trình duyệt xóa cookie bằng cách sử dụng **setMaxAge** để chỉ định tuổi tối đa là 0.

Ví dụ: Tham khảo trong 8.10 Modifying Cookie Values: Tracking User Access Counts và 8.11 Using Cookies to Remember User Preferences

Theo dõi phiên làm việc - Session Tracking

Using basic session tracking

Storing immutable (**nonmodifiable**) **objects** vs. storing mutable (**modifiable**) **objects**

Tracking user access counts

Accumulating user purchases

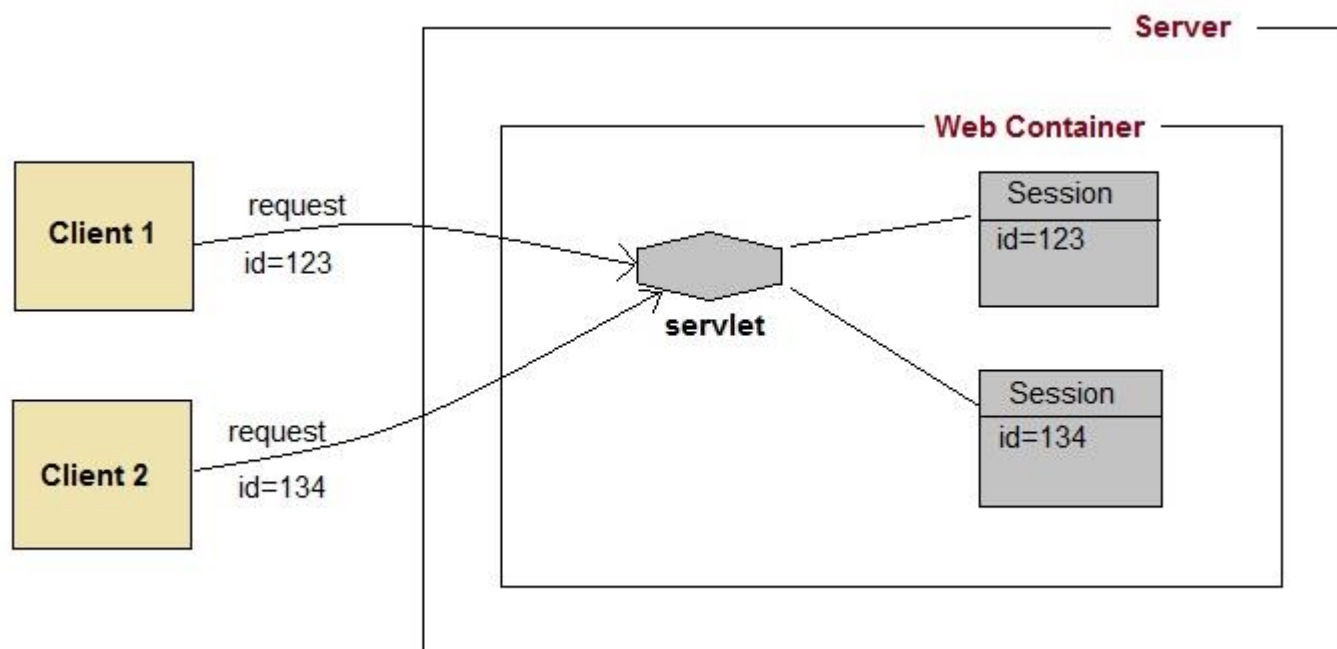
Implementing a shopping cart

Building an online store



Session Tracking (theo dõi phiên làm việc)

Khi user đăng nhập và bắt đầu sử dụng ứng dụng, chúng ta sẽ có thể lấy thông tin duy nhất để xác định phiên làm việc (có thể là user id hoặc session id). Thông tin này sẽ được lưu giữ và đính kèm theo các **request** và **reponse** để xác định user và session. Khi user logout hoặc thoát chương trình, chúng ta có thể xóa thông tin này để kết thúc session.



Session Tracking (theo dõi phiên làm việc) – Lấy thông tin trong session

1. Gọi phương thức **request.getSession** trả về đối tượng kiểu **HttpSession**.
2. Gọi phương thức **getAttribute** của đối tượng **HttpSession** để lấy thông tin trong session:

```
HttpSession session = request.getSession();  
SomeClass someObject = (SomeClass) session.getAttribute("someIdentifier");
```



Session Tracking (theo dõi phiên làm việc) – thiết lập giá trị

Gọi phương thức **setAttribute** của đối tượng HttpSession để lưu trữ thông tin lên session:

```
HttpSession session = request.getSession();
SomeClass someObject = (SomeClass) session.getAttribute("someIdentifier");

if (someObject == null) { // chưa có đối tượng trong session
    someObject = new SomeClass(...);
    session.setAttribute("someIdentifier", value);
}
doSomethingWith(someObject);
```

Cấu trúc dữ liệu không sửa được - **Nonmodifiable** Data Structure

Cấu trúc dữ liệu không sửa được là loại cấu trúc chỉ được tạo không thể sửa đổi. Để thay đổi giá trị của đối tượng như vậy trong session, chúng ta phải thực hiện như sau:

```
HttpSession session = request.getSession();
SomeImmutableClass value =
    (SomeImmutableClass) session.getAttribute("someIdentifier");

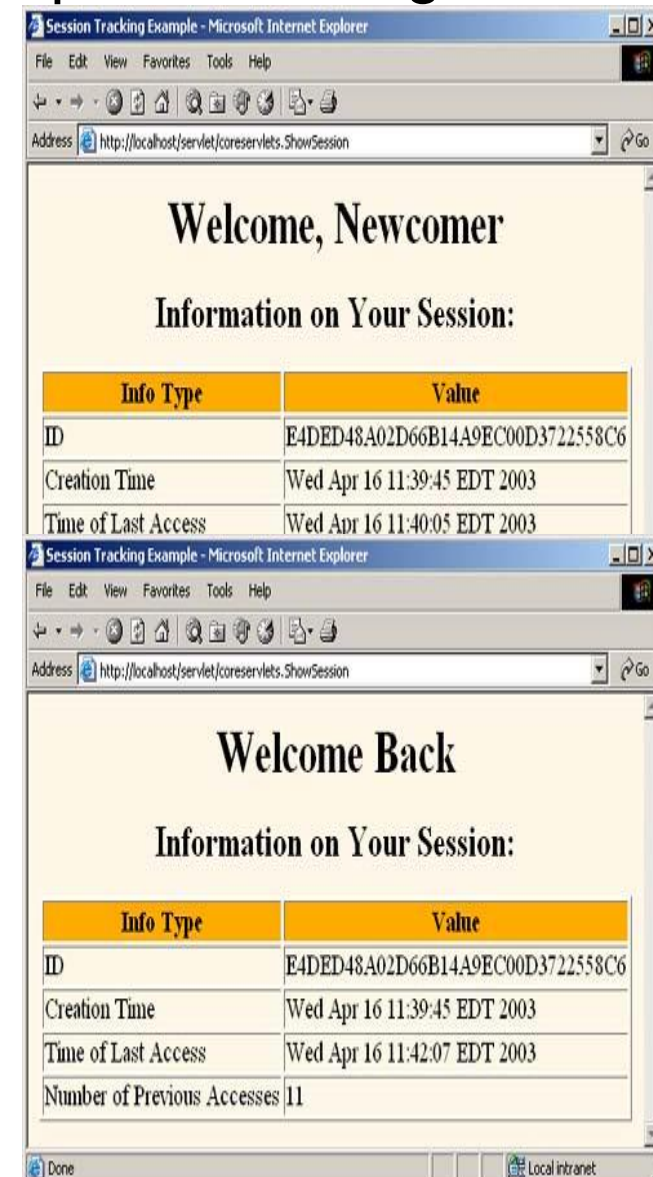
if (value == null) { // Nếu chưa có đối tượng => tạo mới
    value = new SomeImmutableClass (InitValue);
} else { //Nếu đã có đối tượng => cũng phải tạo mới để thay đổi giá trị
    value = new SomeImmutableClass( calculatedFrom(value) );
}
session.setAttribute("someIdentifier", value); //lưu trữ giá trị mới
doSomethingWith(value);
```

Ví dụ về cấu trúc dữ liệu không thể sửa đổi

Một Servlet sử dụng **Integer** để lưu trữ số lần truy cập của một khách hàng. **Integer** là cấu trúc dữ liệu là một cấu trúc **Immutable**.

```
HttpSession session = request.getSession();
Integer accessCount = (Integer)session.getAttribute("accessCount");
if (accessCount == null) {
    accessCount = new Integer(1);
    heading = "Welcome, Newcomer";
} else {
    heading = "Welcome Back";
    accessCount = new Integer(accessCount.intValue() + 1);
}
session.setAttribute("accessCount", accessCount);
```

Tham khảo ví dụ **9.6 A Servlet That Shows Per-Client Access Counts**



Cấu trúc dữ liệu có thể sửa được - Modifiable Data Structure

Cấu trúc dữ liệu có thể sửa được như: **array**, **List**, **Map**, các lớp có thuộc tính **instance**,... Để thay đổi giá trị của đối tượng có cấu trúc modifiable trong session, chúng ta có thể sử dụng phương thức **setAttribute** để thay đổi giá trị theo mẫu sau:

```
HttpSession session = request.getSession();
SomeMutableClass value = (SomeMutableClass)session.getAttribute("someIdentifier");

if (value == null) { // Nếu chưa có đối tượng trong session
    value = new SomeMutableClass(...);
    session.setAttribute("someIdentifier", value);
}
else
    value.updateInternalState(...);

doSomethingWith(value);
```

Ví dụ tích lũy danh sách sản phẩm user chọn trong giỏ hàng

Cấu trúc dữ liệu sử dụng ở đây là **ArrayList** :

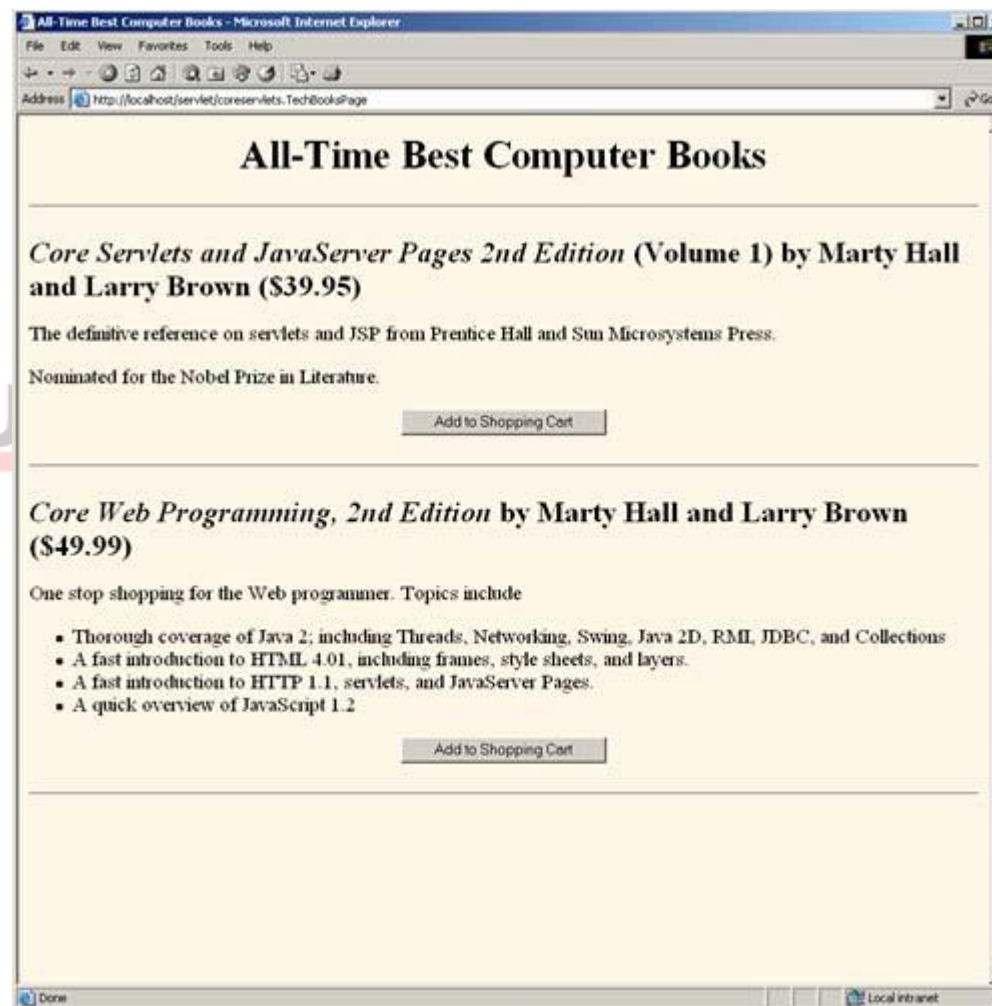
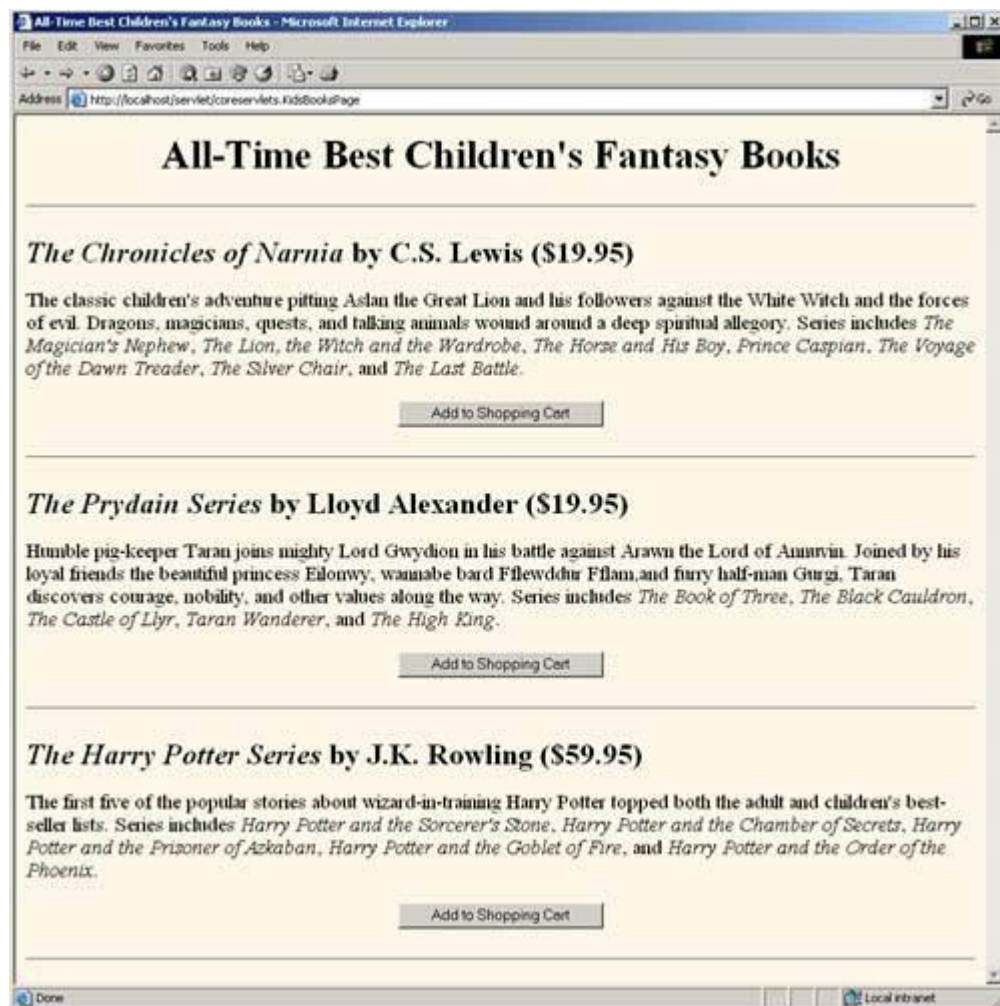
```
HttpSession ses= request.getSession()
ArrayList previousItems = (ArrayList)ses.getAttribute("previousItems");
if (previousItems == null) { // Nếu chưa có
    previousItems = new ArrayList(); // tạo một arraylist rỗng
    session.setAttribute("previousItems", previousItems);
}
String newItem = request.getParameter("newItem");
synchronized (previousItems) {
    if (newItem != null)    previousItems.add(newItem);
    if (previousItems.size() == 0) { out.println("<I>No items</I>");
    } else {
        for(int i=0; i<previousItems.size(); i++) {
            out.println("<LI>" + (String)previousItems.get(i));
        }
    }
}
```

Tham khảo ví dụ **trong 9.7 Accumulating a List of User Data**



Ví dụ xây dựng ứng dụng Online Store

Tham khảo ví dụ 9.8 An Online Store with a Shopping Cart and Session Tracking



Ví dụ xây dựng ứng dụng Online Store

Status of Your Order - Microsoft Internet Explorer

Address: http://localhost/servlet/coreervlets.OrderPage?itemID=alexander001

Status of Your Order

Item ID	Description	Unit Cost	Number	Total Cost
alexander001	The Prydain Series by Lloyd Alexander	\$19.95	1 Update Order	\$19.95



Status of Your Order - Microsoft Internet Explorer

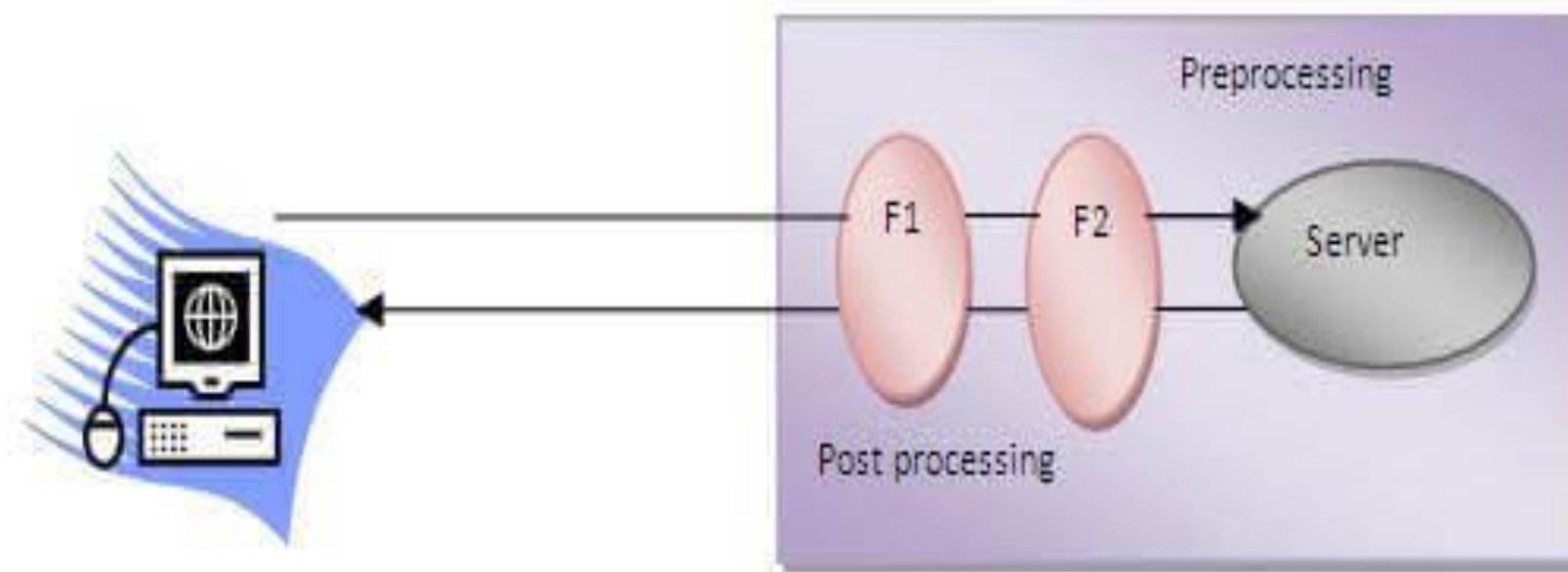
Address: http://localhost/servlet/coreervlets.OrderPage?itemID=hall001&numItems=52

Status of Your Order

Item ID	Description	Unit Cost	Number	Total Cost
alexander001	The Prydain Series by Lloyd Alexander	\$19.95	4 Update Order	\$79.80
rowling001	The Harry Potter Series by J.K. Rowling	\$59.95	1 Update Order	\$59.95
lewis001	The Chronicles of Narnia by C.S. Lewis	\$19.95	1 Update Order	\$19.95
hall001	Core Servlets and JavaServer Pages 2nd Edition (Volume 1) by Marty Hall and Larry Brown	\$39.95	52 Update Order	\$2,077.40
hall002	Core Web Programming, 2nd Edition by Marty Hall and Larry Brown	\$49.99	23 Update Order	\$1,149.77

Bộ lọc - Servlet Filter

Bộ lọc – Servlet Filter là một **servlet** được gọi **tự động** ở thời điểm trước khi, sau khi xử lý yêu cầu.



Ví dụ Basic Filter

index.html

```
<a href="servlet1">click here</a>
```

MyFilter.java

```
import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.*;

public class MyFilter implements Filter{

    public void init(FilterConfig arg0) throws ServletException {}

    public void doFilter(ServletRequest req, ServletResponse resp,
        FilterChain chain) throws IOException, ServletException {

        PrintWriter out=resp.getWriter();
        out.print("filter is invoked before");

        chain.doFilter(req, resp);//sends request to next resource

        out.print("filter is invoked after");
    }
    public void destroy() {}
}
```

HelloServlet.java

```
import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.print("<br>welcome to servlet<br>");

    }
}
```

← → ↺ ⓘ localhost:8080/Chapter1/servlet1

```
filter is invoked before
welcome to servlet
filter is invoked after
```

Cấu hình filter

Ví dụ: packetFilter

Tham khảo:

<https://www.javatpoint.com/servlet-filter>

web.xml

For defining the filter, filter element of web-app must be defined just like servlet.

```
<web-app>

<servlet>
<servlet-name>s1</servlet-name>
<servlet-class>HelloServlet</servlet-class>
</servlet>

<servlet-mapping>
<servlet-name>s1</servlet-name>
<url-pattern>/servlet1</url-pattern>
</servlet-mapping>

<filter>
<filter-name>f1</filter-name>
<filter-class>MyFilter</filter-class>
</filter>

<filter-mapping>
<filter-name>f1</filter-name>
<url-pattern>/servlet1</url-pattern>
</filter-mapping>

</web-app>
```

download this example (developed using Mveclinse IDF)

Ví dụ về Authentication Filter

Chúng ta có thể thực hiện việc **xác thực** trong các bộ lọc. Ví dụ như sau:

```
public void doFilter(ServletRequest req, ServletResponse resp,  
    FilterChain chain) throws IOException, ServletException {  
  
    PrintWriter out=resp.getWriter();  
  
    String password=req.getParameter("password");  
    if(password.equals("admin")){  
        chain.doFilter(req, resp); //sends request to next resource  
    }  
    else{  
        out.print("username or password error!");  
        RequestDispatcher rd=req.getRequestDispatcher("index.html");  
        rd.include(req, resp);  
    }  
}
```

<https://www.javatpoint.com/authentication-filter>

Ví dụ về Filter

Filter thực hiện xóa các **khoảng trắng** trong các tham số (dùng phương thức trim())

```
public void doFilter(ServletRequest request, ServletResponse response,
    FilterChain chain)
    throws IOException, ServletException {
    System.out.println("Filter");
    Enumeration enum = request.getParameterNames();
    while (enum.hasMoreElements()) {
        String parameterName = (String) enum.nextElement();
        String parameterValue = request.getParameter(parameterName);
        request.setAttribute(parameterName, parameterValue.trim());
    }

    chain.doFilter(request, response);
}
```

Tham khảo Book 2 – Chapter 7: A Filter that Checks User Input

Thao tác với CSDL

1. Tạo bảng dữ liệu
2. Tạo form nhập liệu
3. Tạo các lớp: *Emp.java (model); EmpDao.java; SaveServlet, EditServlet,...*

localhost:9765/CrudServlet x

localhost:9765/CrudServlet/ViewServlet

[Add New Employee](#)

Employees List

Id	Name	Password	Email	Country	Edit	Delete
63	Amit Kumar	amtkmj45	amitkumar@gmail.com	India	edit	delete
61	Rahul Kumar	rahul4000	rahulkk@gmail.com	India	edit	delete
62	Sonoo Jaiswal	sonoobsk	sonoojaiswal1987@gmail.com	India	edit	delete
44	adarsh kumar	kkkkk	adarsh232@gmail.com	India	edit	delete

localhost:9765/CrudServlet x

localhost:9765/CrudServlet/EditServlet?id=63

Update Employee

Name:

Password:

Email:

Country:

<https://www.javatpoint.com/crud-in-servlet>

Xây dựng **An Online Store** sử dụng DB

Tham khảo ví dụ 9.8 An Online Store with a Shopping Cart and Session Tracking
Result of the KidsBooksPage servlet Result of the TechBooksPage servlet

